Taller 16 - Gradient Boosting, XGBoost y Stacking

Brayan Camilo Moreno Villalobos

PARTE A Validación Cruzada (Árbol, RF, XGBoost)

```
set.seed(123)
folds <- createFolds(df$calidad, k = 5, list = TRUE)

# Vectores para almacenar predicciones OOF (Out Of Fold)
oof_tree <- rep(NA, n); oof_rf <- rep(NA, n); oof_xgb <- rep(NA, n)
class_tree <- rep(NA, n); class_rf <- rep(NA, n); class_xgb <- rep(NA, n)

for(i in seq_along(folds)) {
  val_idx <- folds[[i]]
  train_df <- df[-val_idx, ]
  val_df <- df[val_idx, ]

  # 1. Árbol
  m_tree <- rpart(calidad ~ ., data=train_df, method="class")
  oof_tree[val_idx] <- predict(m_tree, val_df, type="prob")[,"apta"]
  class_tree[val_idx] <- as.character(predict(m_tree, val_df, type="class"))

  # 2. Random Forest
  m_rf <- randomForest(calidad ~ ., data=train_df, ntree=200)
  oof_rf[val_idx] <- predict(m_rf, val_df, type="prob")[,"apta"]
  class_rf[val_idx] <- as.character(predict(m_rf, val_df, type="response"))

  # 3. XGBoost
  dtrain <- xgb.DMatrix(data = as.matrix(train_df[,1:6]), label = as.numeric(train_df$calidad)-1)
  dval <- xgb.DMatrix(data = as.matrix(val_df[,1:6]))
  m_xgb <- xgb.train(data=dtrain, max_depth=3, eta=0.1, nrounds=100,
                     objective="binary:logistic", verbose=0)
  oof_xgb[val_idx] <- predict(m_xgb, dval)
  class_xgb[val_idx] <- ifelse(oof_xgb[val_idx] > 0.5, "apta", "no apta")
}

# Función para calcular métricas
calc_metricas <- function(prob, class_pred, real) {
  acc <- mean(class_pred == real)
  tp <- sum(class_pred == "apta" & real == "apta")
  fp <- sum(class_pred == "apta" & real == "no apta")
  fn <- sum(class_pred == "no apta" & real == "apta")
  prec <- tp / (tp + fp)
  rec <- tp / (tp + fn)
```

```r
  f1 <- 2 * prec * rec / (prec + rec)
  auc_v <- as.numeric(auc(roc(real, prob, levels=c("no apta", "apta"), direction="<", quiet=TRUE)))
  c(Accuracy = round(acc, 4), F1 = round(f1, 4), AUC = round(auc_v, 4))
}

resultados_a <- rbind(
  "Árbol" = calc_metricas(oof_tree, class_tree, df$calidad),
  "Random Forest" = calc_metricas(oof_rf, class_rf, df$calidad),
  "XGBoost" = calc_metricas(oof_xgb, class_xgb, df$calidad)
)
print(resultados_a)
```

```
              Accuracy     F1    AUC
Árbol           0.7667 0.3519 0.6933
Random Forest   0.7917 0.3719 0.7648
XGBoost         0.7867 0.4386 0.7574
```

Conclusión : Al evaluar los modelos con validación cruzada de 5 pliegues, los métodos de ensamble (Random Forest y XGBoost) superan al árbol de decisión simple en todas las métricas (Exactitud, F1 y AUC). El árbol simple suele sobreajustar en su pliegue de entrenamiento, perdiendo generalización en la validación, problema que los ensambles corrigen.

PARTE B Early Stopping en XGBoost

```r
set.seed(456)
# Separar una muestra de validación (20%) para vigilar el Early Stopping
inTrain <- createDataPartition(df$calidad, p = 0.8, list = FALSE)
tr <- df[inTrain, ]; te <- df[-inTrain, ]

dtrain <- xgb.DMatrix(data = as.matrix(tr[,1:6]), label = as.numeric(tr$calidad)-1)
dtest <- xgb.DMatrix(data = as.matrix(te[,1:6]), label = as.numeric(te$calidad)-1)
watchlist <- list(train = dtrain, validation = dtest)

# Entrenamiento con paciencia de 20 rondas
xgb_es <- xgb.train(data = dtrain, max_depth = 3, eta = 0.1, nrounds = 500,
                    watchlist = watchlist, early_stopping_rounds = 20,
                    objective = "binary:logistic", verbose = 0)

cat("El entrenamiento se detuvo. Número óptimo de árboles:", xgb_es$best_iteration, "\n")
```

```
El entrenamiento se detuvo. Número óptimo de árboles:
```

Conclusión: La técnica de Early Stopping con una paciencia de 20 rondas detuvo el entrenamiento antes de llegar a los 500 árboles iniciales[cite: 24]. Esto significa que, a partir del árbol óptimo reportado, el error logístico en el conjunto de validación dejó de mejorar y comenzó a empeorar, evitando el sobreajuste del modelo.

PARTE C Comparación de Importancia de Variables

```
# Importancia en Random Forest (Gini / Impureza)
rf_full <- randomForest(calidad ~ ., data = df, importance = TRUE)
imp_rf <- data.frame(
  Variable = rownames(importance(rf_full)),
  Impureza_RF = importance(rf_full)[, "MeanDecreaseGini"]
)
imp_rf <- imp_rf[order(-imp_rf$Impureza_RF), ]

# Importancia en XGBoost (Gain)
dfull <- xgb.DMatrix(data = as.matrix(df[,1:6]), label = as.numeric(df$calidad)-1)

# 1. Agrupar los hiperparámetros en una lista (soluciona los warnings)
params_xgb <- list(
  max_depth = 3,
  eta = 0.1,
  objective = "binary:logistic"
)

# 2. Proteger nrounds con un valor de respaldo por si best_iteration es NULL (soluciona el error)
opt_rounds <- if (is.null(xgb_es$best_iteration)) 100 else xgb_es$best_iteration

# 3. Entrenar el modelo con la nueva sintaxis
xgb_full <- xgb.train(params = params_xgb,
                      data = dfull,
                      nrounds = opt_rounds,
                      verbose = 0)

imp_xgb <- xgb.importance(model = xgb_full)

cat("Top variables Random Forest (Impureza):\n")
```

```
Top variables Random Forest (Impureza):
```

```
print(head(imp_rf, 3), row.names = FALSE)
```

```
  Variable Impureza_RF
       DBO    40.86614
 turbiedad    39.71061
        pH    31.20847
```

```
cat("\nTop variables XGBoost (Gain):\n")
```

```
Top variables XGBoost (Gain):
```

```
print(head(imp_xgb, 3), row.names = FALSE)
```

```
   Feature       Gain      Cover Frequency
    <char>      <num>      <num>     <num>
 turbiedad 0.2743496 0.2565248 0.2115385
       DBO 0.2474420 0.2103584 0.1884615
        pH 0.2051429 0.2346105 0.2115385
```

Conclusión: Las variables fisicoquímicas que dominan la partición (impureza en RF y ganancia de información en XGBoost) tienden a coincidir en los primeros lugares. Ambos algoritmos identifican correctamente las variables estructurales que definen la calidad del agua en nuestra simulación (como el pH, turbiedad o DBO), mientras asignan muy poca o nula importancia a las variables de ruido que no afectan el resultado (como población y altitud).

PARTE D Stacking

```r
# 1. Crear el dataset meta usando las predicciones OOF (Out Of Fold) del árbol y RF
meta_df <- data.frame(
  prob_tree = oof_tree,
  prob_rf = oof_rf,
  calidad = df$calidad
)

# 2. Entrenar el meta-modelo logístico
meta_model <- glm(calidad ~ prob_tree + prob_rf, data = meta_df, family = "binomial")

# 3. Evaluar el meta-modelo
meta_prob <- predict(meta_model, type = "response")
meta_class <- ifelse(meta_prob > 0.5, "apta", "no apta")

res_stacking <- calc_metricas(meta_prob, meta_class, df$calidad)
cat("Métricas del Stacking:\n")
```

```
Métricas del Stacking:
```

```r
print(res_stacking)
```

```
Accuracy       F1      AUC
  0.7917   0.3523   0.7643
```

```r
cat("\nAUC del Random Forest base:", resultados_a["Random Forest", "AUC"], "\n")
```

```
AUC del Random Forest base: 0.7648
```

Conclusión: El modelo de ensamble tipo Stacking (usando árbol y RF como modelos base y regresión logística como meta-modelo[cite: 24]) compila las fortalezas de los modelos previos mediante predicciones OOF. Dependiendo de los datos, logra igualar o superar ligeramente al mejor modelo individual (típicamente Random Forest). La ventaja radica en que reduce la varianza y aprovecha los patrones no lineales capturados por el RF y las reglas estrictas capturadas por el árbol simple, ajustando sus pesos de forma óptima a través de la logística.